

"This is my opinion, if you feel you have a better way please go ahead"

3-DAY EXECUTION GUIDE: SPRING BOOT BACKEND & REACT INTEGRATION

6-Developer Blueprint • Java 21 • Spring Boot 3 • Maven • Persistent H2 (`./data`) • Swagger UI • GitHub Copilot

1. Team Setup & Package-by-Feature Structure (Day 1 - Morning)

To avoid Git merge conflicts across 6 developers, divide responsibilities and strictly isolate code by feature domain:

- **Dev 1 (Foundation Lead):** Sets up Maven baseline, H2 config, CORS, Swagger UI, and Global Exception Handling.
- **Devs 2 to 6 (Domain Owners):** Own dedicated feature vertical slices (e.g., Auth, User, Product, Order, Billing, Analytics).

```
src/main/java/com/app/
├── config/           # Dev 1: WebConfig (CORS), Swagger
├── common/           # Dev 1: GlobalExceptionHandler, ApiResponse
└── modules/          # Isolated vertical slices
    ├── auth/         # Dev 1: Controller, Service, Repository, Entity, DTO
    ├── user/         # Dev 2
    ├── product/      # Dev 3
    ├── order/        # Dev 4
    ├── billing/      # Dev 5
    └── analytics/    # Dev 6
```

2. Maven, Persistent H2 & CORS Configuration (Day 1 - Morning by Dev 1)

`application.properties` (stored in `src/main/resources/`):

```
server.port=8080

# Persistent H2 file in project root ./data folder
spring.datasource.url=jdbc:h2:file:./data/appdb;AUTO_SERVER=TRUE;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

springdoc.swagger-ui.path=/swagger-ui.html
```

- **Git Guardrail:** Add `/data/` to `.gitignore` so binary database files are never committed to Git.
- **CORS:** Configure `WebMvcConfigurer` allowing `http://localhost:3000` and `http://localhost:5173` for `GET`, `POST`, `PUT`, `DELETE`, `PATCH`, `OPTIONS`.

3. Contract Extraction & UI Ceiling (Day 1 - Afternoon)

Rule: The React UI is the ceiling. Do not create backend endpoints or fields that are not visually present on the screen.

Copilot Prompt in React:

```
@workspace #file:MyComponent.jsx Inspect all forms, tables, and states in this component. 1. List only the user interactions present (table load, form submit, delete button) and map to HTTP methods. 2. Generate a canonical JSON request payload (submits) and response payload (display). 3. Strip display formatting: ensure currency amounts are raw numbers (e.g., 49.99) and dates are ISO-8601 strings. 4. Ensure every entity has an explicit unique 'id' field.
```

4. Backend Code Generation with Copilot (Day 2)

Each developer works exclusively inside their `modules/<feature>/` folder:

- **Entity & Repository:**

Copilot Prompt: Using this clean JSON: [paste canonical JSON], generate a Spring Boot 3 JPA @Entity named [EntityName] with @Id @GeneratedValue(strategy = GenerationType.IDENTITY) and validation annotations. Also generate a Spring Data JpaRepository.

- **Service & RestController:**

Copilot Prompt: Generate a Service class with CRUD operations and a @RestController mapped to '/api/v1/[feature]'. Return ResponseEntity with standard status codes (200 for GET/PUT, 201 for POST, 204 for DELETE). Include validation with @Valid. If React uses simple .map(), return List<DTO> (no Page wrapper).

- **Baseline Test Data:** Each developer adds 3–5 SQL `INSERT` statements to `src/main/resources/data.sql` matching the React UI's visual data.

5. Mandatory Swagger Verification Against React Component (Day 3 - Morning)

Before modifying any React code, verify that the Spring Boot output matches the component's expectations:

1. Start the backend: `mvn spring-boot:run`.
2. Open Swagger UI at `http://localhost:8080/swagger-ui.html`.
3. Execute your endpoint and copy the JSON response body.
4. Run this verification prompt in GitHub Copilot:

Copilot Comparison Prompt:

Compare this Swagger API response JSON: [paste Swagger JSON] with the state and JSX inside #file:MyComponent.jsx. 1. Do all field names and casing match exactly? 2. Are data types identical (numbers vs strings)? 3. Does the component expect a raw array or an object wrapper? 4. Flag any mismatches so the Java DTO can be adjusted immediately before wiring React.

6. Step-by-Step API Integration with React Using Copilot (Day 3 - Afternoon)

Follow these 4 sequential steps to replace mock state with live Spring Boot API calls cleanly without breaking UI rendering:

SUB-STEP 6.1 Create Central Axios Client (Dev 1 / Dev 2)

Install Axios (`npm install axios`) and create a centralized instance in `src/api/client.js` :

Copilot Prompt: Create an Axios client instance in `src/api/client.js` with `baseUrl: 'http://localhost:8080'` and standard headers: `{ 'Content-Type': 'application/json' }`. Add a response interceptor that extracts error messages from the backend's `GlobalExceptionHandler` JSON payload.

SUB-STEP 6.2 Generate Domain API Helper Functions

Create a dedicated API helper (e.g., `src/api/productApi.js`) rather than putting raw Axios calls directly inside JSX:

Copilot Prompt: Based on the Swagger endpoints for `[FeatureName]`, write helper functions in `src/api/[feature]Api.js` using our `apiClient`: 1. `get[Feature]List()` -> `GET /api/v1/[feature]` 2. `get[Feature]ById(id)` -> `GET /api/v1/[feature]/{id}` 3. `create[Feature](data)` -> `POST /api/v1/[feature]` 4. `update[Feature](id, data)` -> `PUT /api/v1/[feature]/{id}` 5. `delete[Feature](id)` -> `DELETE /api/v1/[feature]/{id}`

SUB-STEP 6.3 Hook Up Component Data Fetching (GET)

Replace hardcoded mock data inside `useState` with an asynchronous `useEffect` lifecycle hook:

Copilot Prompt: `#file:MyComponent.jsx` Refactor this component to use `get[Feature]List()` from `#file:[feature]Api.js`. 1. Replace initial mock state with an empty array `[]`. 2. Fetch data inside a `useEffect` hook on mount. 3. Add a loading state (show spinner/skeleton) and error state (show error alert). 4. Preserve all existing JSX layout, styling classes, and rendering logic.

SUB-STEP 6.4 Hook Up User Actions (POST, PUT, DELETE)

Convert local state mutation handlers (forms and buttons) into asynchronous backend calls:

Copilot Prompt: `#file:MyComponent.jsx` Update the form `onSubmit` and delete button click handlers: 1. For submit: Call `create[Feature](formData)`. On HTTP 201 success, append new item to state or re-fetch list, clear form, and show success toast. 2. For delete: Call `delete[Feature](id)`. On HTTP 204 success, filter deleted item out of state. 3. Handle HTTP 400 validation errors by displaying backend error messages next to the inputs.

7. Dead Code Pruning & End-to-End Verification (Day 3 - Late Afternoon)

• Dead Code Pruning:

Copilot Prompt: Compare `#file:MyComponent.jsx` with `#file:MyController.java`. Are there any backend endpoints or DTO fields that the React component never uses? List them so they can be removed.

- **End-to-End Test:** Verify data displays from `./data`, submit new forms, and test that invalid inputs trigger clear HTTP 400 error toasts on the UI.